Utilizzo di dati COPERNICUS

Premessa ed esempio d'uso

I dati satellitari osservati sono su Copernicus Data Space Ecosystem
https://documentation.dataspace.copernicus.eu/Data/SentinelMissions/Sentinel5P.html

I dati modellistici CAMS (Copernicus Atmosphere Monitoring Service) sono analisi e anche previsioni (a 4 giorni) coprono gli inquinanti NO2, O3 e CO (le PM sono pure disponibili ma comportano una modellazione ulteriore.
Sono in Atmosphere Data Store https://ads.atmosphere.copernicus.eu
cercare per esempio CAMS European Air Quality Forecasts

I files hanno formato **NetCDF (.nc)**

I dati Copernicus sull'inquinamento atmosferico (compresi NO_2, $PM_{2.5}$, ozono, SO_2, CO, ecc.) sono **liberamente accessibili a chiunque**, senza distinzione tra cittadini, università, aziende o enti pubblici. Copernicus adotta una politica di accesso **"free, full and open"** per la quasi totalità dei suoi dati e servizi. .

I dati sono diffusi tramite:

- Copernicus Data Space Ecosystem per i prodotti Sentinel;
- Atmosphere Data Store (ADS) di CAMS per analisi, re-analisi e previsioni atmosferiche;
- ⚠️⚠️⚠️ **API e toolbox Python** per download automatici e integrazione in workflow scientifici.

Per leggere i files .nc si può utilizzare il software gratuito QGIS

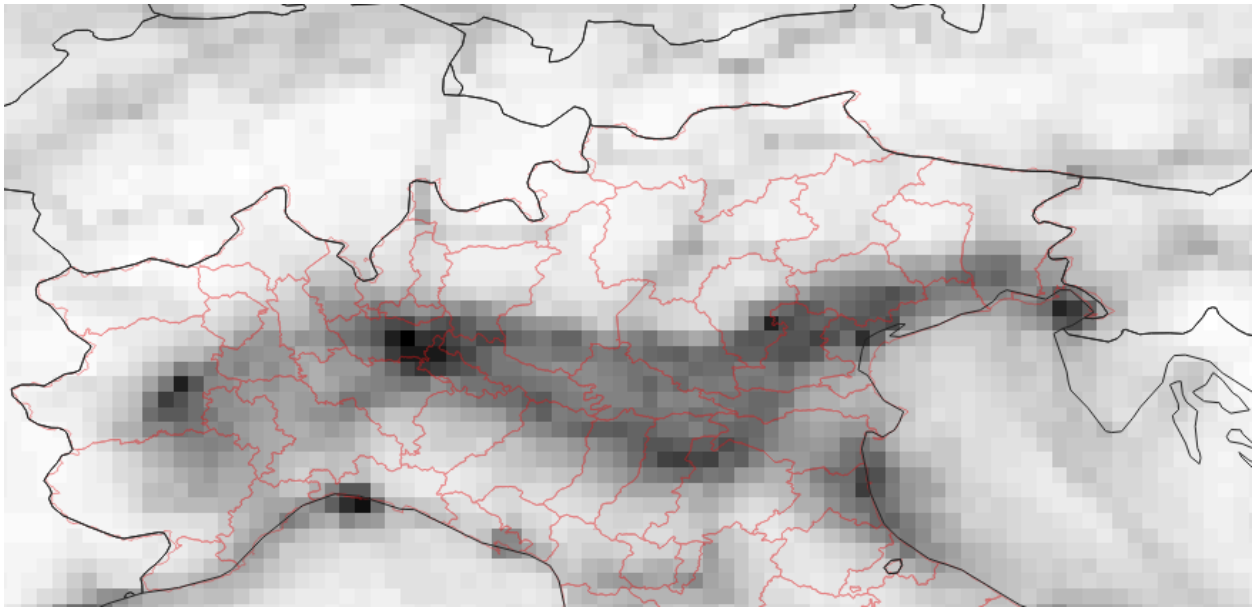
QGIS (Quantum GIS) è un software gratuito e open source per i **Sistemi Informativi Geografici (GIS)**. Serve per visualizzare, analizzare e creare mappe utilizzando dati geografici.

Cosa ho fatto:

1) scaricato una mappa di valori di NO2 con limiti geografici rettangolari per la Pianura Padana
2) scaricato una mappa raster per i confini degli stati dal sito

3) dal sito https://www.naturalearthdata.com ho scaricato la mappa **Admin 0 – Countries** in formato shape .shp
4) dal sito ISTAT basi territoriali e cartografia (https://www.istat.it/it/archivio/222527) ho scaricato Regioni Italiane con Province sempre .shp
5) I layer vanno ridefiniti nella trasparenza e colori dei bordi per sovrapporre le immagini in modo leggibile
6) Poi i colori di default dei livelli di biossido di azoto li ho ridefiniti come toni di grigio

Il risultato di puro esempio è questo:



Account free personale su Copernicus e libreria API

registrarsi come privato su ECMWF

https://cds.climate.copernicus.eu/

Ottenuta la registrazione nel proprio profilo si può accedere ad una API key per accessi automatizzati.

Copernicus offre due sistemi API, il vecchio **cdsapi** ed il nuovo (consigliabile) **ecmwf client**

che si installa come libreria Python nell'ambiente uv con il comando:

```
uv add --upgrade ecmwf-datastores-client
```

Il client può leggere l'url da un file di configurazione personale

```
~/.ecmwfdatastoresrc
```

(bisogna poi capire come far funzionare la cosa in una function app se dovesse servire)

questo è il contenuto del file

```
url: https://ads.atmosphere.copernicus.eu/api
key: LA_TUA_API_KEY
```

per verificare che funzioni eseguire questo comando di accesso, se c'è una risposta allora l'autenticazione è ok

```
uv run python -c "from ecmwf.datastores import Client; c=Client(); print(c.check_authentication())"
```

la risposta che si ottiene è tipo questa:

```
{'id': 5****0, 'sub': '6******2-2**4-4**2-a**9-b**********5', 'role': None, 'email': 'tiziano.terranova@gmail.com'}
```

verifica dei parametri ammessi nella richiesta

```
from ecmwf.datastores import Client

client = Client()

dataset = "cams-europe-air-quality-reanalyses"

constraints = client.apply_constraints(
    dataset,
    {}
)

print(constraints)
```

fornisce la seguente risposta: (formatto per leggibilità)

```
{
    'variable':
        ['ammonia', 'carbon_monoxide', 'dust', 'formaldehyde',
'glyoxal', 'nitrogen_dioxide', 'nitrogen_monoxide',
'non_methane_vocs', 'ozone', 'particulate_matter_10um',
'particulate_matter_2.5um', 'peroxyacyl_nitrates',
'pm10_sea_salt_dry', 'pm10_wildfires', 'pm2.5_ammonium',
'pm2.5_nitrate', 'pm2.5_sulphate', 'pm2.5_total_organic_matter',
'residential_elementary_carbon', 'secondary_inorganic_aerosol',
'sulphur_dioxide', 'total_elementary_carbon'],
    'model':
        ['chimere', 'dehm', 'emep', 'ensemble', 'euradim', 'gemaq',
'lotos', 'match', 'minni', 'mocage', 'monarch', 'silam'],
    'level':
        ['0', '100', '1000', '2000', '250', '3000', '50', '500',
'5000', '750'],
    'type':
        ['interim_reanalysis', 'validated_reanalysis'],
    'year':
        ['2013', '2014', '2015', '2016', '2017', '2018', '2019',
'2020', '2021', '2022', '2023', '2024', '2025'],
    'month':
        ['01', '02', '03', '04', '05', '06', '07', '08', '09', '10',
'11', '12']
}
```

Scarico dei dati per la sola regione emilia-romagna (bound latitudine e longitudine)
mese di gennaio del 2023

```
# Copernico - Test CAMS
# download dei dati CAMS (Copernicus Atmosphere
Monitoring Service)
# per l'ozono (ozone) per il mese di gennaio 2023.
# dati ristretti all'area geografica compresa
# tra latitudine 45.2 e 44.0 e longitudine 9.0 e 13.0
(Italia settentrionale).

from ecmwf.datastores import Client

client = Client()

dataset = "cams-europe-air-quality-reanalyses"

request = {
    "variable": ["ozone"],
    "model": ["ensemble"],
    "level": ["0"],
    "type": ["validated_reanalysis"],
    "year": ["2023"],
    "month": ["01"],
    "area": [45.2, 9.0, 44.0, 13.0],
}

target = "o3_2023_01.nc.zip"

print("Avvio download...")
client.retrieve(dataset, request, target)

print(f"Download completato: {target}")
```

il file scaricato è uno zip in realtà non è compresso ma contiene il nome del vero file

```
unzip <nome_file.zip>
```

Il file è .nc significa **NetCDF** è possibile vedere il tipo di file con il comando ncdump

su linux si installa con

```shell
sudo apt update
sudo apt install netcdf-bin
ncdump --version
```

il tipo di file si verifica anche con

```
$ file cams.eaq.vra.ENSa.o3.l0.2023-01.area-subset.45.2.13.0.44.0.9.0.nc
cams.eaq.vra.ENSa.o3.l0.2023-01.area-subset.45.2.13.0.44.0.9.0.nc: Hierarchical Data Format (version 5) data
```

Un **Hierarchical Data Format (HDF)** è un formato di file pensato per memorizzare **grandi quantità di dati scientifici organizzati in modo strutturato**.
è utile perché ci sono molte misure che riferiscono ad ogni punto di coordinate geografiche (si evita di usare un tabellone, un po' come si risparmia spazio nella strutturazione dimensionale di un DB.

Addendum: cosa sono i termini di model per lo scarico dei dati:

Termine	Significato	In breve
CHIMERE	*CHIMERE chemistry-transport model*	Modello di trasporto e chimica atmosferica sviluppato principalmente in Francia
DEHM	*Danish Eulerian Hemispheric Model*	Modello euleriano danese per la simulazione della chimica atmosferica
EMEP	*European Monitoring and Evaluation Programme* / modello EMEP	Sistema/modello europeo per il trasporto e la chimica degli inquinanti atmosferici
Ensemble	Combinazione dei diversi modelli	Media/integrazione delle previsioni dei vari modelli CAMS
EURAD-IM	*EURopean Air pollution Dispersion – Inverse Model*	Modello europeo di dispersione e chimica atmosferica

GEM-AQ	*Global Environmental Multiscale – Air Quality*	Modello atmosferico globale/regionale con componente di qualità dell'aria
LOTOS-EUROS	*Long Term Ozone Simulation – European Operational Smog model*	Modello europeo per ozono, particolato e altri inquinanti
MATCH	*Multi-scale Atmospheric Transport and CHemistry model*	Modello multiscala di trasporto e chimica atmosferica
MINNI	*MINNI – Italian National Integrated assessment modelling system*	Sistema/modello italiano per la simulazione della qualità dell'aria
MOCAGE	*MOdèle de Chimie Atmosphérique à Grande Échelle*	Modello francese di chimica atmosferica su larga scala
MONARCH	*Multiscale Online Non-hydrostatic AtmospheRe CHemistry model*	Modello multiscala online di meteorologia e chimica atmosferica
SILAM	*System for Integrated modeLling of Atmospheric coMposition*	Modello finlandese per il trasporto e la composizione atmosferica

Conversione da formato .nc NetCDF a tabella PostgreSQL

libreria python xarray

xarray è una libreria Python pensata per lavorare con array multidimensionali associati a coordinate e dimensioni nominate.

In pratica, è molto utile quando hai dati del tipo:

- temperatura nel tempo e nello spazio
- dati meteorologici
- immagini/satelliti
- simulazioni scientifiche
- dati oceanografici
- serie temporali multidimensionali
- file NetCDF, GRIB e altri formati scientifici

Il vantaggio di xarray su numpy.ndarray è che si possono definire i nomi delle dimensioni e delle coordinate, mentre numpy.ndarray usa solo indici numerici

esempio e confronto

```
import xarray as xr
import numpy as np

temperature = xr.DataArray(
    np.random.rand(365, 50, 100),
    dims=["time", "lat", "lon"],
    coords={
        "time": range(365),
        "lat": np.linspace(40, 50, 50),
        "lon": np.linspace(5, 15, 100),
    }
)
```

si possono fare operazioni potenti in modo naturale sui dati

```
# Media nel tempo
temperature.mean(dim="time")

# Media sulla latitudine
temperature.mean(dim="lat")

# Selezionare una certa area
temperature.sel(lat=45, lon=10, method="nearest")

# Selezionare un intervallo temporale
temperature.sel(time=slice(100, 200))
```

libreria python netcdf4

netCDF4 è una libreria Python per leggere, creare e modificare file nel formato NetCDF, molto usato nella ricerca scientifica.

con un file nc per esempio `temperature.nc`

lo si può aprire molto agevolmente con

```
from netCDF4 import Dataset
ds = Dataset("temperature.nc", "r")
```

vedere le variabili contenute

```
print(ds.variables.keys())
```

da netCDF a pandas

```
import xarray as xr
ds = xr.open_dataset("mio_file.nc")
df = ds.to_dataframe()
print(df.head())
```

in questo modo si può ottenere un multiindice

```
                        temperature  pressure
time        lat   lon
2025-01-01  40.0  10.0       12.4      1012
2025-01-01  40.0  11.0       13.1      1010
2025-01-01  41.0  10.0       11.8      1013
...
```

se si vogliono colonne separate si deve usare

```
df = ds.to_dataframe().reset_index()
```

in questo modo si ottiene

```
         time   lat   lon   temperature   pressure
0   2025-01-01  40.0  10.0       12.4        1012
1   2025-01-01  40.0  11.0       13.1        1010
2   2025-01-01  41.0  10.0       11.8        1013
```

consiglio: guardare cosa contiene il file

```
import xarray as xr
```

```
ds = xr.open_dataset("mio_file.nc")
print(ds)
```

si può ottenere per esempio

```
<xarray.Dataset>
Dimensions:      (time: 365, lat: 100, lon: 200)
Coordinates:
  * time         (time) datetime64[ns]
  * lat          (lat) float64
  * lon          (lon) float64

Data variables:
    temperature  (time, lat, lon) float32
    pressure     (time, lat, lon) float32
```

i numeri associati alle dimensioni danno un'idea della grandezza del dataset, nel nostro caso 365x100x200 sono 7,3 milioni di righe!!!

per convertire in pandas

```
import xarray as xr
ds = xr.open_dataset("mio_file.nc")
df = ds.to_dataframe().reset_index()
print(df.head())
```

a questo punto si fa tutto, si salva in csv o si carica come tabella postgres.

Forecast - previsioni a medio termine

libreria cdsapi

installazione → `uv add cdsapi`

test → `uv run python -c "import cdsapi; print(cdsapi.__version__)"`

NOTA IMPORTANTE: l'esempio fornito sul sito ECMWF è incompleto e dà errore nell'esecuzione perché dimentica il parametro "data" che và specificato nel dictionary "request"

Questo è un esempio dello script python che funziona nel download, specifica anche il nome del file. In ogni caso il file contenuto nell'archivio ZIP si chiama sempre `ENS_FORECAST.nc`

```
# Copernico - CAMS - pipeline forecast pm10
# download dei dati CAMS (Copernicus Atmosphere
Monitoring Service)
# dati ristretti all'area geografica compresa
# tra latitudine 45.2 e 44.0 e longitudine 9.0 e 13.0
(Italia settentrionale).

import cdsapi

dataset = "cams-europe-air-quality-forecasts"

request = {
    "variable": ["particulate_matter_10um"],
    "model": ["ensemble"],
    "level": ["0"],
    "type": ["forecast"],
    "date": ["2026-08-30"],
    "time": ["00:00"],
    "leadtime_hour": ["0"],
    "data_format": "netcdf_zip",
    "area": [45.2, 9, 43.7, 13]
}

client = cdsapi.Client()
client.retrieve(dataset,
request).download("pm10-forecast.nc.zip")
```

come nel caso netCDF anche la libreria cdsapi richiede nella $HOME la presenza di un file

`.cdsapirc` che contiene url e key di accesso

```
url: https://ads.atmosphere.copernicus.eu/api
key: 3******7-2**2-4**9-8**0-4*********9c
```

lettura del file **forecast.nc**, conversione in data frame **pd** e caricamento in **postgres**

```
# lettura del file .nc e conversione in dataframe pandas

import xarray as xr
import pandas as pd

ds = xr.open_dataset("ENS_FORECAST.nc")

df = ds.to_dataframe().reset_index()

# salva in file csv
df.to_csv("ENS_FORECAST.csv", index=False)

# salva in tabella golden.forecast_pm10
from sqlalchemy import create_engine, text
from dotenv import load_dotenv
import os

load_dotenv()
user = os.getenv("DB_USER")
password = os.getenv("DB_PASSWORD")
host = os.getenv("DB_HOST")
port = os.getenv("DB_PORT")
db = os.getenv("DB_NAME")

DATABASE_URL =
f"postgresql+psycopg2://{user}:{password}@{host}:{port}/{
db}"

engine = create_engine(DATABASE_URL)

df.to_sql(
    "forecast_pm10",
    engine,
    schema= "gold",
    if_exists="replace",
    index=False
)
```

In Voltaire dalla tabella gold.forecast_pm10 si crea la view gold.forecast_comuni_pm10
questa serve per avere le previsioni associate al singolo comune, facendo join attraverso il reticolo geografico.
nello script function app va quindi cancellata prima e ricreata dopo la sostituzione della tabella forecast_pm10

cancellazione prima:

```
#   cancellazione vecchia view forecast_comunipm10
    logging.info("inizio cancellazione vecchia view forecast_comuni_pm10")
    with engine.begin() as conn:
        conn.execute(text("DROP VIEW if exists gold.forecast_comuni_pm10;"))
    logging.info("cancellata la vecchia view forecast_comuni_pm10")
```

creazione poi:

```
# creazione della view forecast_comuni_pm10
    logging.info("inizio creazione view forecast_comuni_pm10")
    with engine.begin() as conn:
        conn.execute(text("""
            create view gold.forecast_comuni_pm10 as (
                with forecast as (
                    select
                          current_date::timestamp + time * INTERVAL '1 hour' as ora,
                          latitude as lat, longitude as lon,
                          p.lat_index, p.lon_index,
                           pm10_conc as pm10
                    from gold.forecast_pm10 fp
                    left join lateral silver.punto_griglia(fp.latitude::numeric, fp.longitude::numeric) p
                    ON TRUE
                    )
                SELECT cg.comune, cg.sigla_provincia, f.ora, f.pm10
                FROM silver.comuni_geogrid cg
                JOIN forecast f
                ON cg.lat_index = f.lat_index AND cg.lon_index = f.lon_index
                );
            """))
```

funzione di utilità per calcolare il punto griglia approssimato

```
-- creazione della funzione SQL di calcolo del punto griglia approssimato
-- date le coordinate geografiche del punto qualunque interno alla griglia
-- l'intenso uso di ::NUMERIC è per evitare errori di arrotondamento
CREATE OR REPLACE FUNCTION silver.punto_griglia(
   p_lat NUMERIC,
   p_lon NUMERIC
)
RETURNS TABLE (
   lat_grid NUMERIC,
   lon_grid NUMERIC,
   lat_index INTEGER,
   lon_index INTEGER
)
LANGUAGE SQL
IMMUTABLE
AS $$
   WITH indici AS (
       SELECT
           ROUND((p_lat - 43.75::NUMERIC) / 0.1::NUMERIC)::INTEGER AS i,
           ROUND((p_lon - 9.05::NUMERIC)  / 0.1::NUMERIC)::INTEGER AS j
   )
   SELECT
       43.75::NUMERIC + i * 0.1::NUMERIC,
       9.05::NUMERIC  + j * 0.1::NUMERIC,
       i,
       j
   FROM indici
   WHERE i BETWEEN 0 AND 14
     AND j BETWEEN 0 AND 39;
$$;
```